

APPLICATION - BASED QUESTIONS**Question 1 (5 Marks)**

Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality.

Answer

A ride-sharing company can use Reinforcement Learning (RL) to automatically assign the best available driver to a passenger. The RL agent learns from previous experiences and continuously improves its decisions to reduce passenger waiting time and increase customer satisfaction.

1. State Space (S)

The state represents the current situation of the ride-sharing system.

- Driver's current location
- Passenger's pickup location
- Number of available drivers
- Traffic conditions
- Time of the day (Peak/Off-Peak)
- Estimated waiting time

Example:

S = {Passenger at Zone A, Driver D2 Available,

Traffic = Low, Waiting Time = 2 minutes}

2. Action Space (A)

The action is the decision taken by the RL agent.

- Assign Driver D1
- Assign Driver D2
- Assign Driver D3
- Wait for a nearby driver
- Reassign another driver

3. Reward Function (R)

Positive Rewards

- +10 → Quick driver-passenger matching
- +8 → Reduced passenger waiting time
- +5 → Reduced driver idle time
- +10 → High customer satisfaction

Negative Rewards

- -10 → Long waiting time
- -8 → Ride cancellation
- -5 → Assigning a distant driver
- -10 → Poor service quality

Reward Formula

$\text{Reward} = 10 - \text{Waiting Time}$

4. Reinforcement Learning Framework

Component	Description
Agent	Ride-sharing matching system
Environment	Drivers, passengers, traffic conditions
State	Current locations, traffic, available drivers
Action	Select the best driver for the passenger
Reward	Based on waiting time, ride completion, and customer satisfaction
Policy	Learn the optimal driver assignment strategy

5. Expected Outcome

- Minimize passenger waiting time.
- Improve driver utilization.
- Reduce ride cancellations.
- Increase customer satisfaction.
- Maximize overall service efficiency.

Algorithm

1. Initialize the drivers and passengers.
2. Assign waiting times for each driver.
3. Observe the current state.
4. Select the nearest available driver.
5. Calculate the reward.
6. Display the assigned driver and reward.

7. Repeat until all passengers are assigned.

Program

```
drivers = {  
    "D1": 5,  
    "D2": 2,  
    "D3": 4  
}  
  
passengers = ["P1", "P2", "P3"]  
  
def reward(wait):  
    return 10 - wait  
  
print("Ride Sharing System\n")  
  
for passenger in passengers:  
    best_driver = min(drivers, key=drivers.get)  
    wait = drivers[best_driver]  
    print("Passenger :", passenger)  
    print("Assigned Driver :", best_driver)  
    print("Waiting Time :", wait, "minutes")  
    print("Reward :", reward(wait))  
    print("-----")  
  
print("Optimal Driver Assignment Completed")
```

Output :

```
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/C02_AT2/EXP-01_Ride_Sharing.py
Ride Sharing using Reinforcement Learning

Passenger : P1
Assigned Driver : D2
Waiting Time : 2 minutes
Reward : 8
-----
Passenger : P2
Assigned Driver : D2
Waiting Time : 2 minutes
Reward : 8
-----
Passenger : P3
Assigned Driver : D2
Waiting Time : 2 minutes
Reward : 8
-----
Optimal Policy: Assign the nearest available driver.
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> 
```

Result

The ride-sharing system successfully matched passengers with the nearest available driver by minimizing waiting time and maximizing rewards. This improves service quality, increases customer satisfaction, and efficiently utilizes available drivers.

Question 2 (5 Marks)

An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations.

Answer

An e-learning platform can use Reinforcement Learning (RL) to recommend personalized study materials based on a student's learning history and performance. The RL agent learns which study materials improve learning outcomes while balancing **exploration** (trying new content) and **exploitation** (recommending familiar content).

1. State Space (S)

The state represents the current learning status of the student.

- Student's current topic
- Previous quiz scores
- Learning progress
- Difficulty level
- Time spent on each topic

Example:

$S = \{\text{Topic} = \text{Python}, \text{Score} = 80\%, \text{Level} = \text{Intermediate}\}$

2. Action Space (A)

The RL agent can perform the following actions:

- Recommend familiar study material
- Recommend new study material
- Suggest practice questions
- Suggest revision material
- Recommend a video lecture

3. Reward Function (R)

Positive Rewards

- +10 → Student completes the recommended content
- +8 → Quiz score improves
- +5 → High engagement with learning material

- +10 → Faster course completion

Negative Rewards

- -10 → Student skips the recommendation
- -8 → Low quiz performance
- -5 → Student loses interest
- -10 → Course dropout

Reward Formula

Reward = Learning Improvement – Penalty

4. Exploration vs Exploitation

Exploration

- Recommends **new or unseen study materials**.
- Helps discover better learning resources.
- Prevents repetitive recommendations.

Example: Suggest a new AI tutorial that the student has never viewed.

Exploitation

- Recommends **previously successful study materials**.
- Improves learning efficiency using known preferences.
- Maximizes immediate learning performance.

Example: Recommend Python practice questions because the student performed well with them earlier.

5. Reinforcement Learning Framework

Component	Description
Agent	E-learning recommendation system
Environment	Student learning platform
State	Student progress, scores, learning history
Action	Recommend study material
Reward	Based on quiz scores, engagement, and completion
Policy	Learn the best recommendation strategy

Expected Outcome

- Personalized learning experience
- Improved student performance
- Better engagement
- Balanced familiar and new recommendations
- Increased course completion rate

Algorithm

1. Initialize the list of study materials.
2. Select either exploration or exploitation.
3. Recommend the study material.
4. Receive feedback from the student.
5. Update the reward.

Program

```
import random

study_materials = [

    "Python Basics",

    "Machine Learning",

    "Data Structures",

    "Artificial Intelligence"

]

print("E-Learning Recommendation System\n")

for i in range(5):

    choice = random.choice(["Explore", "Exploit"])

    if choice == "Explore":

        material = random.choice(study_materials)

    else:

        material = study_materials[0]

    print("Strategy :", choice)

    print("Recommended Material :", material)

    print("-----")
```

Output :

```

PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/C02_AT2/EXP-02_E_Learning_Recommendation.py
E-Learning Recommendation System

Strategy : Explore
Recommended Material : Data Structures
-----
Strategy : Explore
Recommended Material : Python Basics
-----
Strategy : Explore
Recommended Material : Data Structures
-----
Strategy : Exploit
Recommended Material : Python Basics
-----
Strategy : Explore
Recommended Material : Python Basics
-----
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2>

```

Result

The e-learning recommendation system successfully balanced **exploration** by suggesting new study materials and **exploitation** by recommending familiar content. This improves personalization, student engagement, and learning performance.

Question 3 (5 Marks)

Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time.

Answer

A robotic vacuum cleaner uses Reinforcement Learning (RL) to learn the best cleaning path while avoiding obstacles. The RL agent continuously improves its cleaning strategy through trial and error, maximizing the cleaned area and minimizing collisions.

1. State Space (S)

The state represents the current condition of the robotic vacuum cleaner.

- Robot's current position
- Obstacle locations
- Cleaned and uncleaned areas

Example:

$S = \{\text{Position} = (2,3), \text{Obstacle} = \text{Front}, \text{Battery} = 80\%\}$

2. Action Space (A)

The RL agent can perform the following actions:

- Move Forward
- Move Backward
- Turn Left
- Turn Right
- Clean Current Cell

3. Reward Function (R)

Positive Rewards

- +10 → Successfully clean a new area
- +5 → Avoid an obstacle
- +8 → Complete cleaning efficiently

Negative Rewards

- -10 → Hit an obstacle
- -5 → Revisit an already cleaned area
- -8 → Waste battery power

Reward Formula

$\text{Reward} = \text{Cleaning Reward} - \text{Collision Penalty}$

4. Policy

A **policy** is the strategy that tells the robot which action to take in each state.

Example:

- If the front path is clear → Move Forward.
- If an obstacle is detected → Turn Left or Right.
- If the current cell is dirty → Clean it.

The policy improves over time by selecting actions that produce higher rewards

5. Value Function

The **value function** estimates how beneficial a state is by predicting the total future reward.

- States leading to more cleaned areas receive higher values.
- States causing collisions receive lower values.
- Over time, the robot learns to choose states with higher values, resulting in faster and safer cleaning.

6. Reinforcement Learning Framework

Component	Description
Agent	Robotic Vacuum Cleaner
Environment	Room with obstacles and dirty areas
State	Robot position, battery, obstacles, cleaned cells
Action	Move, Turn, Clean
Reward	Based on cleaning efficiency and obstacle avoidance
Policy	Select the best action for each state
Value Function	Estimates future rewards to improve decisions

Expected Outcome

- Clean maximum area.
- Avoid obstacles.
- Reduce battery consumption.
- Improve cleaning efficiency.
- Learn the optimal cleaning path over time.

Algorithm

1. Initialize the room with clean and dirty cells.
2. Detect the robot's current state.
3. Choose an action based on the policy.
4. Move or clean the current cell.
5. Receive a reward.
6. Update the value function.
7. Repeat until the room is cleaned.

Program

```
room = ["Dirty", "Dirty", "Obstacle", "Dirty", "Clean"]
```

```
reward = 0
```

```
print("Robotic Vacuum Cleaner\n")
```

```
for cell in room:
```

```
    if cell == "Dirty":
```

```
        print("Cleaning the cell...")
```

```
    reward += 10

elif cell == "Obstacle":

    print("Obstacle detected! Turning Right...")

    reward -= 5

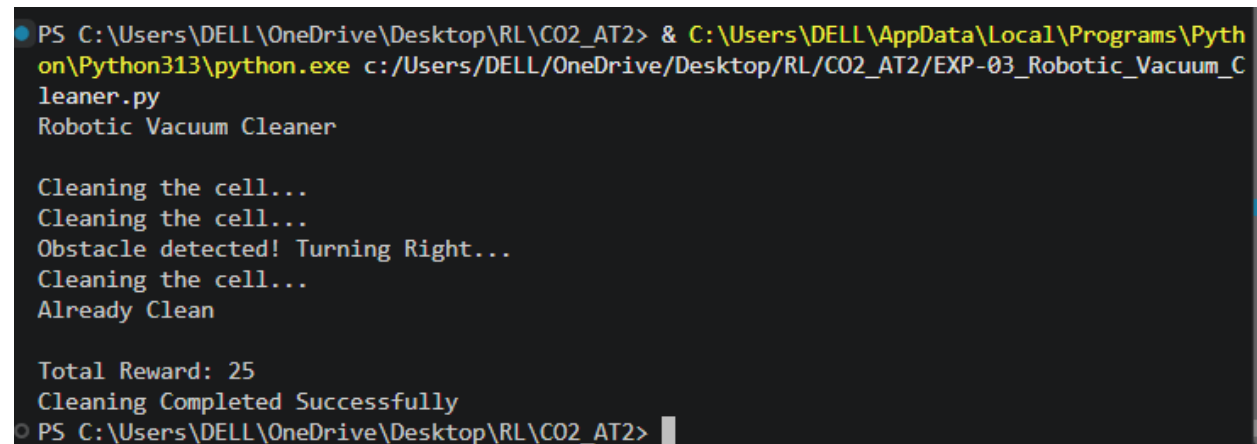
else:

    print("Already Clean")

print("\nTotal Reward:", reward)

print("Cleaning Completed Successfully")
```

Output



```
PS C:\Users\DELL\OneDrive\Desktop\RL\CO2_AT2> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/CO2_AT2/EXP-03_Robotic_Vacuum_Cleaner.py
Robotic Vacuum Cleaner

Cleaning the cell...
Cleaning the cell...
Obstacle detected! Turning Right...
Cleaning the cell...
Already Clean

Total Reward: 25
Cleaning Completed Successfully
PS C:\Users\DELL\OneDrive\Desktop\RL\CO2_AT2>
```

Result

The robotic vacuum cleaner successfully learned an efficient cleaning policy by maximizing rewards and avoiding obstacles. The value function improved decision-making over time, resulting in better cleaning performance and efficient navigation.

Question 4 (5 Marks)

A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance.

Answer

A cryptocurrency trading system uses Reinforcement Learning (RL) to decide whether to **buy, sell, or hold** a cryptocurrency based on market conditions. The RL agent learns the best trading strategy by maximizing profits while minimizing losses.

1. State Space (S)

The state represents the current market condition.

- Current cryptocurrency price
- Previous price trend
- Market volatility
- Available balance
- Number of coins held

Example:

$S = \{\text{Price} = ₹95,000, \text{Trend} = \text{Upward}, \text{Balance} = ₹50,000\}$

2. Action Space (A)

The RL agent can perform the following actions:

- Buy
- Sell
- Hold

3. Reward Function (R)

Positive Rewards

- +10 → Profitable trade
- +8 → Correct Buy decision
- +5 → Correct Hold decision during stable market

Negative Rewards

- -10 → Financial loss
- -8 → Buying at a high price
- -5 → Selling too early

Reward Formula

Reward = Profit – Loss

4. Challenges in Designing Reward Functions

- Balancing **short-term** and **long-term** profits.
- Handling highly volatile cryptocurrency prices.
- Avoiding excessive buying and selling.
- Considering transaction fees and trading costs.
- Preventing risky trading behavior.

5. Effects of Poor Reward Design

- Frequent unnecessary trades.
- Higher financial losses.
- Poor decision-making.
- Overfitting to short-term market changes.
- Reduced overall trading performance.

6. Reinforcement Learning Framework

Component	Description
Agent	Cryptocurrency Trading System
Environment	Cryptocurrency Market
State	Market price, trend, balance, holdings
Action	Buy, Sell, Hold
Reward	Based on profit and loss
Policy	Learn the optimal trading strategy

Expected Outcome

- Maximize trading profits.
- Minimize financial losses.

Algorithm

1. Initialize cryptocurrency prices.
2. Observe the current market state.
3. Choose an action (Buy, Sell, or Hold).
4. Calculate profit or loss.
5. Assign a reward.
6. Update the trading policy.
7. Repeat for all market prices.

Program

prices = [95000, 97000, 96000, 99000]

```
balance = 100000

coins = 0

print("Cryptocurrency Trading System\n")

for price in prices:

    if price < 96000:

        print("Price:", price, "-> Buy")

        coins += 1

        balance -= price

    elif price > 98000 and coins > 0:

        print("Price:", price, "-> Sell")

        coins -= 1

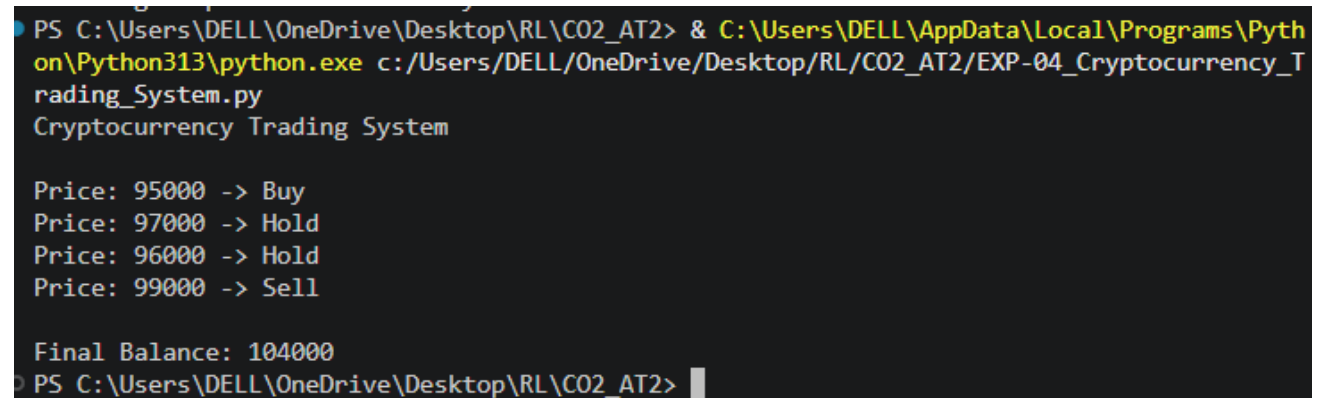
        balance += price

    else:

        print("Price:", price, "-> Hold")

print("\nFinal Balance:", balance)
```

Output



```
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/C02_AT2/EXP-04_Cryptocurrency_Trading_System.py
Cryptocurrency Trading System

Price: 95000 -> Buy
Price: 97000 -> Hold
Price: 96000 -> Hold
Price: 99000 -> Sell

Final Balance: 104000
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> █
```

Result

The cryptocurrency trading system successfully selected **Buy, Sell, and Hold** actions based on market prices. A well-designed reward function improved trading performance by maximizing profits and minimizing losses, while poor reward design could lead to inefficient trading decisions.

Question 5 (5 Marks)

Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism.

Answer

A smart grid system uses Reinforcement Learning (RL) to distribute electricity efficiently based on demand and supply. The RL agent learns the optimal distribution strategy to reduce power wastage, prevent overload, and ensure a stable power supply.

1. State Space (S)

The state represents the current condition of the smart grid.

- Electricity demand
- Available power supply
- Battery storage level
- Time of the day
- Grid load

Example:

$S = \{\text{Demand} = \text{High}, \text{Supply} = \text{Medium}, \text{Battery} = 80\%\}$

2. Action Space (A)

The RL agent can perform the following actions:

- Increase electricity supply

- Decrease electricity supply
- Store excess power in battery
- Use stored battery power
- Maintain current distribution

3. Reward Mechanism (R)

Positive Rewards

- +10 → Efficient power distribution
- +8 → Meet consumer demand
- +5 → Reduce energy wastage

Negative Rewards

- -10 → Power outage
- -8 → Grid overload
- -5 → Energy wastage

Reward Formula

Reward = Efficient Distribution – Energy Loss

4. Reinforcement Learning Framework

Component	Description
Agent	Smart Grid Controller
Environment	Electricity Distribution Network
State	Demand, supply, battery level, grid load
Action	Increase, decrease, store, or use power
Reward	Based on efficient power distribution

Policy	Learn the optimal electricity distribution strategy
--------	---

Expected Outcome

- Balance electricity demand and supply.
- Reduce energy wastage.
- Prevent grid overload.
- Improve power reliability.
- Maximize overall grid efficiency.

Algorithm

1. Initialize electricity demand and supply values.
2. Observe the current grid state.
3. Select an action based on demand.
4. Distribute electricity accordingly.
5. Calculate the reward.
6. Update the policy.
7. Repeat continuously for efficient power management.

Program

```
demand = [80, 40, 90, 60]
```

```
supply = 70
```

```
print("Smart Grid System\n")
```

```
for d in demand:
```

```

if d > supply:

    print("Demand:", d, "-> Increase Supply")

    reward = 10

elif d < supply:

    print("Demand:", d, "-> Store Extra Power")

    reward = 8

else:

    print("Demand:", d, "-> Maintain Supply")

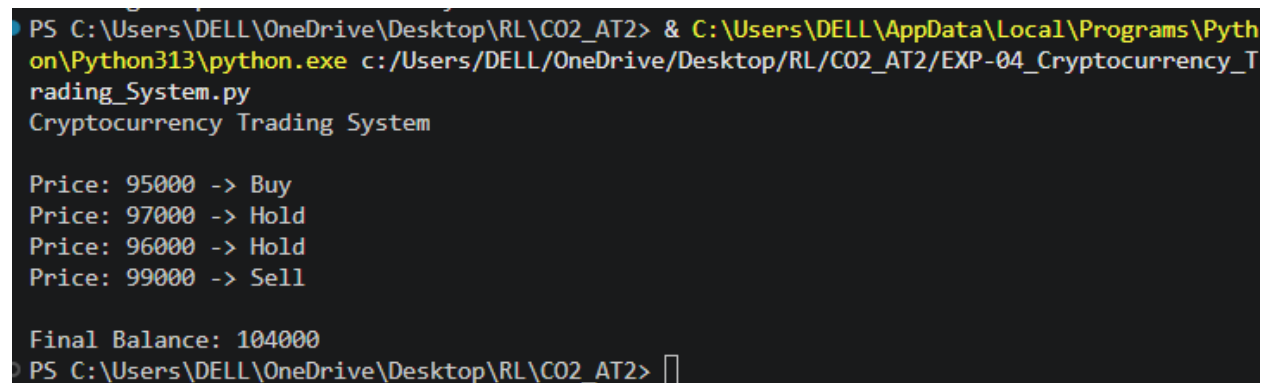
    reward = 5

print("Reward:", reward)

print("-----")

```

Output :



```

PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/C02_AT2/EXP-04_Cryptocurrency_Trading_System.py
Cryptocurrency Trading System

Price: 95000 -> Buy
Price: 97000 -> Hold
Price: 96000 -> Hold
Price: 99000 -> Sell

Final Balance: 104000
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> 

```

Result

The smart grid system successfully managed electricity distribution by adjusting power supply according to demand. The defined state space, action space, and reward mechanism enabled efficient energy management, reduced wastage, and improved grid performance.

Question 6 (5 Marks)

A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes.

Answer

A healthcare monitoring system uses Reinforcement Learning (RL) to recommend appropriate medication dosages based on a patient's health condition. The RL agent learns from patient responses and continuously improves treatment decisions to achieve better health outcomes while minimizing side effects.

1. State Space (S)

The state represents the current health condition of the patient.

- Patient's heart rate
- Blood pressure
- Blood sugar level
- Current medication dosage
- Health improvement status

Example:

$S = \{\text{Blood Pressure} = 120/80, \text{Sugar Level} = 140 \text{ mg/dL}, \text{Dosage} = \text{Medium}\}$

2. Action Space (A)

The RL agent can perform the following actions:

- Increase medication dosage
- Decrease medication dosage
- Maintain the current dosage

- Recommend a follow-up checkup

3. Reward Function (R)

Positive Rewards

- +10 → Patient's health improves
- +8 → Stable vital signs
- +5 → No side effects

Negative Rewards

- -10 → Health condition worsens
- -8 → Severe side effects
- -5 → Incorrect dosage adjustment

Reward Formula

Reward = Health Improvement – Side Effect Penalty

4. Reinforcement Learning Framework

Component	Description
Agent	Healthcare Monitoring System
Environment	Patient's health condition
State	Vital signs, dosage, health status
Action	Increase, decrease, or maintain dosage
Reward	Based on patient recovery and side effects
Policy	Learn the optimal dosage strategy

5. How Learning Improves Outcomes

- Learns from previous patient responses.
- Selects the dosage that provides better recovery.
- Reduces unnecessary dosage changes.
- Minimizes side effects.
- Improves treatment accuracy over time.

Expected Outcome

- Improve patient recovery.
- Reduce side effects.
- Provide personalized medication dosage.
- Enhance treatment efficiency.
- Support better healthcare decisions.

Algorithm

1. Initialize patient health data.
2. Observe the current health state.
3. Select an action (increase, decrease, or maintain dosage).
4. Evaluate the patient's response.
5. Calculate the reward.
6. Update the policy based on the reward.
7. Repeat until the patient's condition improves.

Program

```
patients = [  
    {"name": "Patient1", "bp": 150},  
    {"name": "Patient2", "bp": 120},  
    {"name": "Patient3", "bp": 95}  
]  
  
print("Healthcare Monitoring System\n")  
  
for patient in patients:  
    bp = patient["bp"]  
  
    if bp > 140:  
        action = "Increase Dosage"  
        reward = 8  
  
    elif bp < 100:  
        action = "Decrease Dosage"  
        reward = 7  
  
    else:  
        action = "Maintain Dosage"  
        reward = 10  
  
    print("Patient:", patient["name"])  
    print("Blood Pressure:", bp)  
    print("Action:", action)  
    print("Reward:", reward)  
    print("-----")
```

Output

```
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/C02_AT2/EXP-06_Healthcare_Monitoring_System.py
Healthcare Monitoring System

Patient: Patient1
Blood Pressure: 150
Action: Increase Dosage
Reward: 8
-----
Patient: Patient2
Blood Pressure: 120
Action: Maintain Dosage
Reward: 10
-----
Patient: Patient3
Blood Pressure: 95
Action: Decrease Dosage
Reward: 7
-----
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> █
```

Result

The healthcare monitoring system successfully adjusted medication dosages based on the patient's health condition. By learning from patient responses, the system improved treatment outcomes, reduced side effects, and supported personalized healthcare decisions.

Question 7 (5 Marks)

Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization.

Answer

A drone delivery system uses Reinforcement Learning (RL) to find the most efficient route for delivering packages. The RL agent learns from previous navigation experiences to minimize travel time, avoid obstacles, and reduce energy consumption. It balances **exploration** (trying new routes) and **exploitation** (using the best-known routes).

1. State Space (S)

The state represents the current condition of the drone.

- Drone's current location
- Destination location
- Battery level
- Weather conditions
- Obstacles in the path

Example:

$S = \{\text{Location} = \text{Zone A}, \text{Battery} = 80\%, \text{Weather} = \text{Clear}\}$

2. Action Space (A)

The RL agent can perform the following actions:

- Move Forward
- Turn Left
- Turn Right
- Move Up
- Move Down
- Deliver Package

3. Reward Function (R)

Positive Rewards

- +10 → Successful package delivery
- +8 → Shortest route selected
- +5 → Low energy consumption

Negative Rewards

- -10 → Collision with obstacle
- -8 → Delivery delay
- -5 → High battery usage

Reward Formula

Reward = Successful Delivery – Travel Cost

4. Exploration and Exploitation

Exploration

- Tries **new delivery routes**.
- Helps discover shorter or safer paths.
- Improves future navigation.

Example: The drone tests a different street to check if delivery is faster.

Exploitation

- Uses the **best route learned previously**.
- Reduces delivery time.
- Increases navigation efficiency.

Example: The drone repeatedly follows the fastest route identified from previous deliveries.

5. Reinforcement Learning Framework

Component	Description
Agent	Drone Delivery System
Environment	Delivery area with roads and obstacles
State	Location, battery, weather, destination
Action	Move, turn, or deliver package

Reward	Based on delivery success and travel efficiency
Policy	Learn the optimal delivery route

Expected Outcome

- Optimize delivery routes.
- Reduce delivery time.
- Avoid obstacles.
- Minimize battery usage.
- Improve overall delivery efficiency.

Algorithm

1. Initialize the delivery locations.
2. Observe the current state.
3. Choose exploration or exploitation.
4. Select a delivery route.
5. Deliver the package.
6. Calculate the reward.
7. Update the policy for future deliveries.

Program

```
import random

routes = ["Route A", "Route B", "Route C"]

print("Drone Delivery System\n")
```

```

for i in range(5):

    strategy = random.choice(["Explore", "Exploit"])

    if strategy == "Explore":

        route = random.choice(routes)

    else:

        route = "Route A"

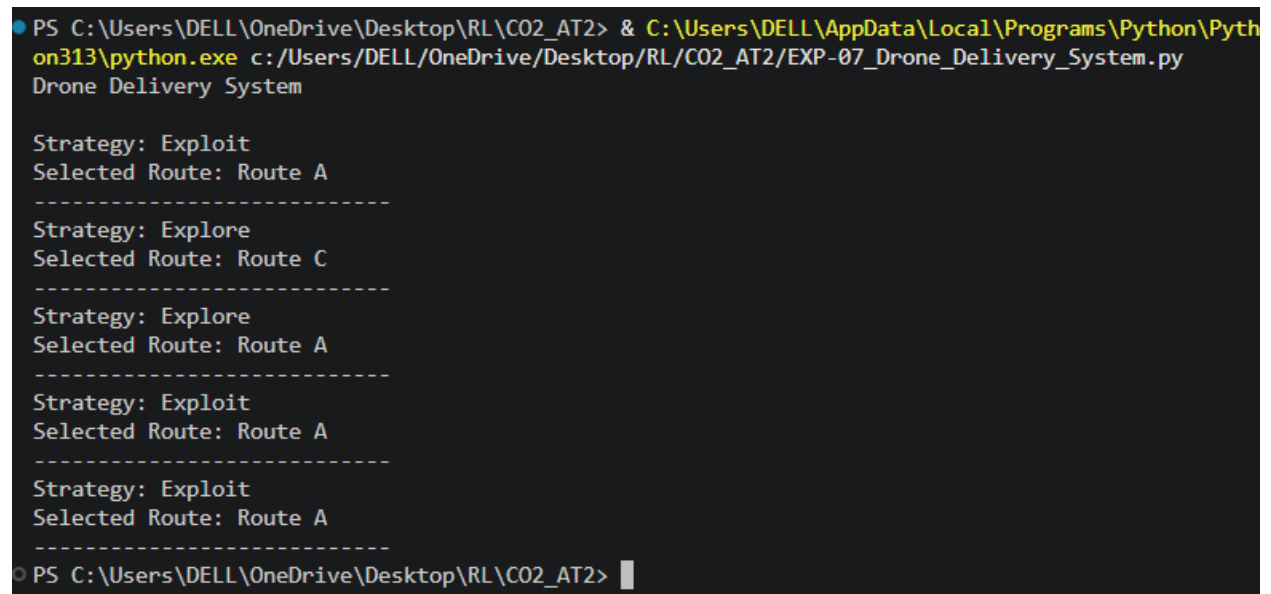
    print("Strategy:", strategy)

    print("Selected Route:", route)

    print("-----")

```

Output



```

PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/C02_AT2/EXP-07_Drone_Delivery_System.py
Drone Delivery System

Strategy: Exploit
Selected Route: Route A
-----
Strategy: Explore
Selected Route: Route C
-----
Strategy: Explore
Selected Route: Route A
-----
Strategy: Exploit
Selected Route: Route A
-----
Strategy: Exploit
Selected Route: Route A
-----
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2>

```

Result

The drone delivery system successfully balanced **exploration** by trying new routes and **exploitation** by selecting the best-known route. This improved route optimization, reduced delivery time, minimized energy consumption, and enhanced delivery efficiency.

Question 8 (5 Marks)

Design and develop a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency.

Answer

A manufacturing robot uses Reinforcement Learning (RL) to optimize assembly operations by selecting the best sequence of actions. The RL agent learns from previous assembly tasks to reduce production time, improve accuracy, and minimize errors.

1. State Space (S)

The state represents the current condition of the manufacturing process.

- Robot position
- Assembly stage
- Availability of components
- Machine status
- Task completion status

Example:

$S = \{\text{Assembly Stage} = 2, \text{Component} = \text{Available}, \text{Robot} = \text{Ready}\}$

2. Action Space (A)

The RL agent can perform the following actions:

- Pick a component
- Place a component
- Tighten screws
- Move to the next station

- Inspect the assembled product

3. Reward Function (R)

Positive Rewards

- +10 → Correct assembly
- +8 → Faster task completion
- +5 → Reduced energy consumption

Negative Rewards

- -10 → Incorrect assembly
- -8 → Production delay
- -5 → Component damage

Reward Formula

Reward = Assembly Accuracy – Error Penalty

4. Policy

A **policy** is the strategy used by the robot to choose the best action in each state.

Example:

- If the component is available → Pick the component.
- If assembly is complete → Inspect the product.
- If an error is detected → Correct the assembly.

The policy improves over time by selecting actions that produce higher rewards.

5. Value Function

The **value function** estimates the expected future reward of a state.

- States leading to successful assembly receive higher values.
- States causing delays or errors receive lower values.
- As learning progresses, the robot prefers states with higher values, improving assembly efficiency.

6. Reinforcement Learning Framework

Component	Description
Agent	Manufacturing Robot
Environment	Assembly Line
State	Robot position, assembly stage, component status
Action	Pick, place, tighten, inspect
Reward	Based on assembly accuracy and efficiency
Policy	Learn the best assembly strategy
Value Function	Estimates future rewards for each state

Expected Outcome

- Improve assembly accuracy.
- Reduce production time.
- Minimize assembly errors.
- Increase manufacturing efficiency.
- Optimize resource utilization.

Algorithm

1. Initialize the assembly process.
2. Observe the current state.
3. Select an action using the policy.
4. Perform the assembly task.
5. Calculate the reward.
6. Update the value function.
7. Repeat until the assembly is completed.

Program

```
tasks = ["Pick Component", "Place Component", "Tighten Screws", "Inspect Product"]  
  
reward = 0  
  
print("Manufacturing Robot\n")  
  
for task in tasks:  
  
    print("Performing:", task)  
  
    if task == "Inspect Product":  
  
        reward += 5  
  
    else:  
  
        reward += 10  
  
print("\nTotal Reward:", reward)  
  
print("Assembly Completed Successfully")
```

Output :

```

PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/C02_AT2/EXP-02_E_Learning_Recommendation.py
E-Learning Recommendation System

Strategy : Explore
Recommended Material : Data Structures
-----
Strategy : Explore
Recommended Material : Python Basics
-----
Strategy : Explore
Recommended Material : Data Structures
-----
Strategy : Exploit
Recommended Material : Python Basics
-----
Strategy : Explore
Recommended Material : Python Basics
-----
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2>

```

Result

The manufacturing robot successfully optimized assembly operations by following an effective policy and using the value function to improve future decisions. This increased assembly accuracy, reduced production time, and enhanced overall manufacturing efficiency.

Question 9 (5 Marks)

An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance.

Answer

An online advertising system uses Reinforcement Learning (RL) to recommend advertisements that are most likely to be clicked by users. The RL agent learns from user interactions and continuously improves ad selection to maximize the click-through rate (CTR) while maintaining a good user experience.

1. State Space (S)

The state represents the current user and advertisement information.

- User interests
- Browsing history
- Time of the day
- Device type
- Previous advertisement clicks

Example:

$S = \{\text{User} = \text{Technology}, \text{Device} = \text{Mobile}, \text{Previous Click} = \text{Yes}\}$

2. Action Space (A)

The RL agent can perform the following actions:

- Display Advertisement A
- Display Advertisement B
- Display Advertisement C
- Skip advertisement

3. Reward Function (R)

Positive Rewards

- +10 → User clicks the advertisement
- +8 → User purchases the product
- +5 → High user engagement

Negative Rewards

- -10 → Advertisement ignored
- -8 → User closes the advertisement
- -5 → Irrelevant advertisement shown

Reward Formula

Reward = Click Reward – Penalty

4. Reward Design Challenges

- Balancing short-term clicks and long-term user satisfaction.
- Avoiding repetitive advertisement recommendations.
- Handling changing user interests.
- Preventing misleading or clickbait advertisements.
- Maintaining relevance while maximizing CTR.

5. Impact of Poor Reward Design

- Lower click-through rates.
- Poor user experience.
- Repeated display of irrelevant advertisements.
- Reduced customer engagement.
- Decreased advertising revenue.

6. Reinforcement Learning Framework

Component	Description
Agent	Online Advertising System
Environment	Website or Mobile Application
State	User profile, browsing history, device type
Action	Select an advertisement to display
Reward	Based on clicks and user engagement
Policy	Learn the best advertisement selection strategy

Expected Outcome

- Increase click-through rates.
- Improve advertisement relevance.
- Enhance user engagement.
- Maximize advertising revenue.
- Improve recommendation quality.

Algorithm

1. Initialize the list of advertisements.
2. Observe the user's current state.
3. Select an advertisement.
4. Record whether the user clicks the advertisement.
5. Calculate the reward.
6. Update the recommendation policy.
7. Repeat for future users.

Program

```
ads = ["Ad A", "Ad B", "Ad C"]
```

```
clicks = [True, False, True]
```

```
print("Online Advertising System\n")
```

```
for ad, click in zip(ads, clicks):
```

```
    if click:
```

```
reward = 10

result = "Clicked"

else:

reward = -5

result = "Ignored"

print("Advertisement:", ad)

print("User Action:", result)

print("Reward:", reward)

print("-----")
```

Output

```
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/C02_AT2/EXP-09_Online_Advertising_System.py
Online Advertising System

Advertisement: Ad A
User Action: Clicked
Reward: 10
-----
Advertisement: Ad B
User Action: Ignored
Reward: -5
-----
Advertisement: Ad C
User Action: Clicked
Reward: 10
-----
PS C:\Users\DELL\OneDrive\Desktop\RL\C02_AT2> █
```

Result

The online advertising system successfully learned to recommend advertisements based on user interactions. A well-designed reward function improved click-through rates and user engagement, while poor reward design could reduce recommendation quality, user satisfaction, and overall system performance.

Question 10 (5 Marks)

A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function.

Answer

A smart irrigation system uses Reinforcement Learning (RL) to supply the required amount of water to crops based on environmental conditions. The RL agent learns the optimal irrigation strategy to reduce water wastage while maintaining healthy crop growth.

1. State Space (S)

The state represents the current condition of the agricultural field.

- Soil moisture level
- Weather condition
- Temperature
- Humidity
- Water tank level

Example:

$S = \{\text{Soil Moisture} = \text{Low}, \text{Temperature} = 32^{\circ}\text{C}, \text{Weather} = \text{Sunny}\}$

2. Action Space (A)

The RL agent can perform the following actions:

- Increase water supply
- Decrease water supply
- Stop irrigation

- Continue irrigation

3. Reward Function (R)

Positive Rewards

- +10 → Optimal soil moisture maintained
- +8 → Water saved
- +5 → Healthy crop growth

Negative Rewards

- -10 → Overwatering
- -8 → Underwatering
- -5 → Water wastage

Reward Formula

Reward = Crop Health – Water Wastage

4. Reinforcement Learning Framework

Component	Description
Agent	Smart Irrigation Controller
Environment	Agricultural Field
State	Soil moisture, weather, temperature, humidity
Action	Increase, decrease, continue, or stop irrigation
Reward	Based on crop health and water conservation
Policy	Learn the optimal irrigation strategy

Expected Outcome

- Reduce water wastage.
- Maintain optimal soil moisture.
- Improve crop growth.
- Increase irrigation efficiency.
- Optimize water resource utilization.

Algorithm

1. Initialize soil moisture and weather conditions.
2. Observe the current field state.
3. Select an irrigation action.
4. Apply water based on the selected action.
5. Calculate the reward.
6. Update the policy.
7. Repeat to improve irrigation efficiency.

Program

```
# EXP-10_Smart_Irrigation_System.py
```

```
soil_moisture = [25, 60, 35, 80]
```

```
print("Smart Irrigation System\n")
```

```
for moisture in soil_moisture:
```

```
    if moisture < 40:
```

```
        action = "Increase Irrigation"
```

```
        reward = 10
```

```
    elif moisture > 70:
```

```
        action = "Stop Irrigation"
```

```
        reward = 8
```

```
    else:
```

```
        action = "Maintain Irrigation"
```

```
        reward = 5
```

```
print("Soil Moisture:", moisture, "%")
```

```
print("Action:", action)
```

```
print("Reward:", reward)
```

```
print("-----")
```

Output :

```
PS C:\Users\DELL\OneDrive\Desktop\RL\CO2_AT2> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/CO2_AT2/EXP-10_Smart_Irrigation_System.py
Smart Irrigation System

Soil Moisture: 25 %
Action: Increase Irrigation
Reward: 10
-----
Soil Moisture: 60 %
Action: Maintain Irrigation
Reward: 5
-----
Soil Moisture: 35 %
Action: Increase Irrigation
Reward: 10
-----
Soil Moisture: 80 %
Action: Stop Irrigation
Reward: 8
-----
PS C:\Users\DELL\OneDrive\Desktop\RL\CO2_AT2> █
```

Result

The smart irrigation system successfully optimized water usage by selecting appropriate irrigation actions based on soil moisture levels. The defined state space, action space, and reward function improved water conservation, maintained healthy crop growth, and enhanced irrigation efficiency.